NWT QUESTION BANK

Exam-ready answers for Modules 4, 5 and 6

Includes all 23 two-mark questions and all 19 five-mark questions.

Prepared primarily from the supplied NWT Unit/Module 4, 5 and 6 notes. Where the notes were brief, answers use standard course-level explanations; a wording note is included where a question is not fully defined by the notes.

2-MARK QUESTIONS

1. Examine the functions of the Application Layer and infer their role in providing network services to end users.

Answer: The Application Layer is the top layer of the TCP/IP model and provides network services directly to user applications. It supports web access (HTTP/HTTPS), email (SMTP, POP3, IMAP), file transfer (FTP), remote login (Telnet/SSH), name resolution (DNS), and network configuration (DHCP). It translates user needs into application-level communication.

2. Classify the different Application Layer protocols and select the appropriate protocol for a given network application.

Answer: Common protocols include HTTP/HTTPS for web pages, FTP for file transfer, SMTP for sending email, POP3/IMAP for receiving email, DNS for translating domain names to IP addresses, DHCP for assigning IP configuration, Telnet/SSH for remote access, and SNMP for network management. For example, use HTTPS for secure browsing and IMAP when email must stay synchronized across devices.

3. Compare connection-oriented and connectionless protocols, and justify their suitability for different communication scenarios.

Answer: Connection-oriented protocols such as TCP establish a connection before transfer and provide acknowledgements, sequencing, retransmission, and reliable delivery. They suit web transactions and file transfer. Connectionless protocols such as UDP send independent datagrams without connection setup; they have lower delay but no delivery or ordering guarantee, so they suit streaming, voice/video calls, and DNS queries.

4. Assess the performance of TCP and UDP by comparing their advantages and limitations for different networking applications.

Answer: TCP is reliable and ordered, with error recovery, flow control, and congestion control, but connection setup and acknowledgements add overhead and delay. UDP has a small fixed 8-byte header and low latency, and supports broadcast/multicast, but does not guarantee delivery, ordering, or retransmission. TCP suits file downloads and web transactions; UDP suits real-time audio/video and time-sensitive queries.

5. Examine the working of the File Transfer Protocol (FTP) and infer its role in file transmission over a network.

Answer: FTP transfers files between an FTP client and server over TCP. It uses a control connection (normally TCP port 21) for commands and replies and a separate data connection for file contents or directory listings. A user connects and authenticates, issues commands such as upload/download, and closes the session. Traditional FTP is unencrypted, so secure alternatives such as SFTP or FTPS are preferred for sensitive data.

6. Analyze the relationship between the Transport Layer and the Application Layer, and infer how they cooperate to deliver end-to-end communication services.

Answer: The Application Layer defines services and protocols used by applications, such as HTTP, SMTP, and FTP. It passes application data to the Transport Layer, which divides it into segments, uses port numbers to identify processes, and provides either reliable TCP delivery or lightweight UDP delivery. At the receiver, transport data is delivered to the correct application process.

7. Illustrate the services provided by the Transport Layer and explain how they ensure reliable end-to-end communication.

Answer: The Transport Layer provides process-to-process delivery, port addressing, multiplexing/demultiplexing, segmentation and reassembly, error detection, flow control, and congestion control. TCP additionally uses sequence numbers, acknowledgements, checksums, and retransmissions to deliver data in order and recover from loss. UDP provides simpler connectionless delivery without those reliability guarantees.

8. Examine the role of a web browser and infer its significance in accessing web resources with a suitable example.

Answer: A web browser is client software that requests resources from web servers using HTTP/HTTPS and displays the returned HTML, CSS, images, and other content. It interprets HTML, applies CSS, executes JavaScript, and provides navigation, caching, and security features. For example, entering a website address causes the browser to resolve the domain through DNS, request the page, and render the response.

9. Analyze the uses of the Document Object Model (DOM) in developing dynamic web pages.

Answer: The DOM represents an HTML document as a tree of objects that JavaScript can access and change. It is used to update text, attributes, styles, and form content; create or remove elements; and respond to clicks, input, and other events without reloading the page. Examples include form validation, dropdown menus, live search results, and changing a theme when a button is clicked.

10. Examine the structure of the Document Object Model (DOM) and infer its importance in web programming.

Answer: The DOM is a hierarchical tree of nodes. The document is the entry point; the html element contains head and body, which contain elements such as title, headings, paragraphs, links, and lists. Nodes have parent-child and sibling relationships. This standard object model lets scripts find and modify page content consistently across browsers.

11. Discuss the history of the Document Object Model (DOM).

Answer: Early browsers in the 1990s exposed different, non-standard scripting interfaces, often called DOM Level 0. W3C standardized DOM Level 1 in 1998 for accessing and changing HTML/XML structure. DOM Level 2 added event handling, styles, traversal, and ranges; DOM Level 3 added features such as improved XML, keyboard events, and XPath support. Modern DOM APIs are maintained as living web standards by WHATWG/W3C-related work and include querySelector and addEventListener.

12. Compare the different levels of event handling and select the appropriate event-handling mechanism for a web application.

Answer: Inline/intrinsic handling places JavaScript directly in an HTML attribute, for example onclick="showMessage()"; it is easy for small demonstrations but mixes markup and code. DOM property handling assigns a function to an event property such as button.onclick, but typically allows one handler per property. addEventListener() separates behavior from HTML, supports multiple listeners and options, and is preferred for maintainable applications.

13. Discuss real-life examples of using DOM in web pages.

Answer: DOM is used for client-side form validation, such as displaying an error beside an empty email field; updating a cart total when quantities change; opening and closing navigation menus; filtering a product list while typing; adding chat messages without a full refresh; and switching between light and dark themes. JavaScript locates the relevant DOM nodes and changes their text, attributes, classes, or styles.

14. Compare the DOM with the HTML source code and infer their roles in web page rendering.

Answer: HTML source is the original markup text downloaded or written by the developer. The browser parses it into the DOM, an in-memory tree of objects that scripts and browser APIs can inspect and modify. The rendered page reflects the current DOM together with CSS and layout rules; therefore, JavaScript changes may alter the displayed page even when the original source text remains unchanged.

15. Apply the process of creating an XML document for a given data representation.

Answer: To create XML, choose one root element, represent each record using meaningful elements, nest related elements correctly, close every tag, and quote attribute values. XML is case-sensitive and must be well-formed. Example: <students><student id="1"><name>Asha</name><course>AI&DS</course></student></students>. The root is students; each student contains name and course.

16. Examine the DOM tree and analyze the event-handling process in web applications.

Answer: The browser parses HTML into a DOM tree of parent, child, and sibling nodes. A script selects a node, registers an event handler (preferably with addEventListener), and waits for an event such as click, input, load, or submit. When the event occurs, the handler runs and can inspect the event object and update the DOM. Events commonly bubble from the target element toward its ancestors.

17. Classify the different types of web pages (static and dynamic) and infer their suitability for various web applications.

Answer: A static web page displays fixed content delivered from a stored HTML file; it is simple, fast, and suitable for portfolios, brochures, and documentation. A dynamic page changes according to user input, data, time, or server responses, using JavaScript and/or server-side code with a database. Dynamic pages suit social networks, online shops, dashboards, and personalized portals but require more logic and maintenance.

18. Examine the concept of Markup Language and analyze its significance in web technologies.

Answer: A markup language uses tags or annotations to describe the structure, meaning, or presentation of document content. HTML structures web pages, XHTML applies stricter XML rules to HTML-like documents, XML stores and transports structured data, and SGML is a general standard from which HTML historically developed. Markup makes content machine-readable and gives browsers or parsers rules for interpreting documents.

19. Discuss the concept of URL rewriting with an example.

Answer: URL rewriting encodes information such as a session identifier or routing parameter in a URL so the server can identify a request or preserve state. Example: response.encodeURL("profile?user=21") may produce a URL containing a session id when cookies are unavailable. It is useful for servlet session tracking, but identifiers in URLs can leak through browser history, logs, or shared links, so sensitive data should not be placed there.

20. Analyze the features of XHTML and infer its role in modern web development.

Answer: XHTML is HTML reformulated using XML syntax. Its rules include lowercase tags, proper nesting, closing all elements, quoted attribute values, explicit attribute values, and a DOCTYPE declaration. These rules improve consistency and parsing by XML tools. However, XHTML is stricter and less forgiving of errors, and modern websites generally use HTML5; XHTML remains relevant where XML-based workflows are required.

21. Examine the applications of Java Servlets and analyze their role in generating dynamic web content.

Answer: Java Servlets are server-side Java components that process client requests and generate responses dynamically. They can read form data, validate users, access databases using JDBC, manage sessions, set cookies, and return HTML or other content. The web container manages servlet lifecycle and request handling. Servlets are platform-independent and can reuse Java APIs, making them useful for web applications and server-side business logic.

22. Apply the standard structure of an HTML document to develop a basic web page.

Answer: A basic HTML document contains a DOCTYPE declaration, html root, head section with metadata and title, and body with visible content. Example: <!DOCTYPE html><html lang="en"><head><meta charset="UTF-8"><title>My Page</title></head><body><h1>Welcome</h1> This is my web page. </body></html>. The browser parses this structure and renders the body content.

23. Compare XHTML with traditional HTML and justify the advantages of XHTML in web development.

Answer: HTML is generally forgiving of syntax errors, while XHTML follows XML rules strictly. XHTML requires lowercase tags, properly nested and closed elements, quoted attribute values, and explicit attribute values. This improves consistency, maintainability, and compatibility with XML tools, but minor syntax mistakes can cause parsing failures and older browsers may not handle XHTML served as XML well. HTML5 is more common for general modern web development.

5-MARK QUESTIONS

1. Distinguish between Multiplexing and Demultiplexing.

Answer: Multiplexing combines data streams from multiple sources for transmission over one shared channel; a multiplexer (MUX) performs this at the sending side. Demultiplexing separates the combined stream at the receiving side and delivers data to the intended destination or application. Multiplexing improves channel utilization, while demultiplexing uses identifiers such as transport-layer port numbers to distribute data correctly. Examples of multiplexing include FDM, TDM, and WDM.

2. Simplify the principle of congestion control in detail.

Answer: Congestion occurs when the traffic entering a network exceeds the capacity of links or routers, causing queue growth, delay, packet loss, and retransmissions. Congestion control aims to keep offered traffic within network capacity. Common approaches include traffic shaping (leaky bucket/token bucket), admission or resource control, queue management, and end-system feedback. TCP adjusts its sending rate using a congestion window; it increases cautiously when delivery succeeds and reduces the rate when loss or congestion is detected. Effective control improves throughput and prevents congestion collapse.

3. Illustrate the Application Layer protocols in detail.

Answer: Application Layer protocols define how user applications exchange messages. HTTP/HTTPS transfers web resources; HTTPS adds TLS security. FTP transfers files using separate control and data connections. SMTP sends outgoing email, while POP3 downloads incoming email and IMAP synchronizes messages on the server. DNS maps domain names to IP addresses. DHCP automatically supplies IP configuration such as address, subnet mask, gateway, and DNS server. Telnet offers remote terminal access but is unencrypted; SSH is the secure alternative. SNMP monitors and manages network devices.

4. Illustrate the Simple Mail Transfer Protocol (SMTP).

Answer: SMTP is an Application Layer protocol used to send and relay email between a client and mail servers or between mail servers. The sender's mail client submits a message to an outgoing server, which looks up the recipient domain's mail server through DNS and relays the message. SMTP uses commands and replies such as HELO/EHLO, MAIL FROM, RCPT TO, DATA, and QUIT. Common ports are 25 for server relay, 587 for message submission, and 465 for implicit TLS. SMTP sends mail; POP3 or IMAP is used to retrieve it.

5. Elaborate the Dynamic Host Configuration Protocol (DHCP) in detail.

Answer: DHCP automatically assigns hosts their network configuration, reducing manual configuration and address conflicts. A typical IPv4 exchange is DORA: Discover (client broadcasts a request), Offer (server offers an address and settings), Request (client requests the offer), and Acknowledge (server confirms the lease). Options may include subnet mask, default gateway, DNS servers, and lease duration. DHCP uses UDP ports 67 (server) and 68 (client). Administrators should use trusted DHCP servers and protections such as DHCP snooping where supported.

6. Compare MIME and POP3 protocols.

Answer: MIME (Multipurpose Internet Mail Extensions) is an email message format extension that identifies content types and enables non-ASCII text, attachments, images, audio, and multipart messages to be represented in email. POP3 (Post Office Protocol version 3) is a protocol for retrieving email from a mail server to a client, often downloading messages for local use. MIME describes the structure and encoding of message content; POP3 handles mailbox access and retrieval. They are complementary rather than competing protocols.

7. Elaborate the Simple Network Management Protocol (SNMP).

Answer: SNMP is used to monitor and manage routers, switches, servers, printers, and other network devices. Its architecture includes a manager/NMS, managed devices, an SNMP agent on each device, and a Management Information Base (MIB) containing managed objects identified by OIDs. The manager can GET values, GET-NEXT/GET-BULK to retrieve data, and SET configurable values; agents can send TRAP or INFORM notifications. SNMP supports fault, performance, and configuration monitoring. SNMPv3 adds authentication, integrity, and privacy; older versions provide weaker security.

8. Illustrate the syntax, variables, and data types used in JavaScript.

Answer: JavaScript statements are commonly terminated with semicolons, blocks use braces, and identifiers are case-sensitive. Variables are declared with var (function-scoped and redeclarable), let (block-scoped and reassignable), or const (block-scoped and cannot be reassigned). Example: let age = 20; const name = "Asha"; if (age >= 18) { console.log(name); }. JavaScript is dynamically typed. Common types include string, number, bigint, boolean, undefined, symbol, and object; null is a primitive value often taught as a separate type.

9. Analyze XML syntax and structure in detail.

Answer: XML stores structured data using user-defined tags. A well-formed document has exactly one root element, case-sensitive matching start/end tags, proper nesting, quoted attribute values, and legal element names. Empty elements may be written as <item/>. Example: <book id="b1"><title>Networks</title><price>250</price></book>. XML may include a declaration such as <?xml version="1.0" encoding="UTF-8"?> and can be validated against a DTD or XML Schema. XML describes/transports data rather than prescribing how it should look.

10. Explain the seven data types in JavaScript.

Answer: The seven commonly listed JavaScript types are: (1) String—text, e.g. "hello"; (2) Number—integers and floating-point values, e.g. 12 or 2.5; (3) BigInt—very large integers, e.g. 123n; (4) Boolean—true or false; (5) Undefined—a declared variable with no assigned value; (6) Symbol—a unique identifier value; and (7) Object—a collection or reference type, including arrays and functions. Note: null is a primitive value too, although some introductory syllabi present it separately or group it under object due to typeof null returning "object".

11. Illustrate client-side scripting using JavaScript.

Answer: Client-side scripting runs in the user's browser and adds behavior without requiring every interaction to be processed by the server. JavaScript can validate forms, respond to mouse/keyboard events, modify the DOM, show messages, update content asynchronously, and control UI elements. Example: input with id age and a button can call checkAge(); the function reads the age field and updates the result paragraph with "Eligible" or "Not eligible". Client-side validation improves usability but important checks must also be repeated on the server for security.

12. Explain the main functions of the Document Object Model (DOM).

Answer: The DOM lets scripts access and manipulate page structure, content, and style. Selection methods include getElementById(), getElementsByTagName(), querySelector(), and querySelectorAll(). Content can be changed with textContent or innerHTML; attributes with setAttribute(); styles/classes with style and classList; and structure with createElement(), append(), and remove(). Event listeners such as addEventListener() let a page respond to clicks, input, load, and form submission. These functions enable interactive pages without full reloads.

13. Analyze the syntax and semantics of XHTML, and evaluate its advantages and limitations in modern web development.

Answer: XHTML uses HTML-like elements but obeys XML syntax. Tags and attributes are lowercase; every element is closed; elements are correctly nested; attribute values are quoted; minimized attributes are written explicitly (selected="selected"); and a DOCTYPE is declared. Semantics means choosing elements according to their meaning, such as h1 for a heading, p for a paragraph, strong for importance, and ul/ol for lists. Advantages include consistent structure, easier parsing by XML tools, maintainability, and portability. Limitations include strict error handling, migration effort, and less widespread use than HTML5 for ordinary websites.

14. Illustrate the types of HTML documents in detail with suitable examples.

Answer: The question wording is broad; the notes mainly cover HTML versions and document structure. HTML versions include HTML 1.0 (basic headings, paragraphs and links), HTML 2.0 (standardized core features), HTML 3.2 (tables and presentational features), HTML 4.01 (stylesheets and improved accessibility), and HTML5 (semantic elements, audio/video, canvas, and continuing living-standard updates). A standard HTML5 document is: <!DOCTYPE html><html lang="en"><head><meta charset="UTF-8"><title>Example</title></head><body><h1>Title</h1> Content </body></html>. XHTML is a stricter XML-based form, not simply another HTML version.

15. Examine the concept of server-side programming and its significance in web development.

Answer: Server-side programming executes on the web server after receiving a client request. It can validate input, authenticate users, enforce authorization, interact with databases, manage sessions, apply business rules, and generate personalized HTML or JSON responses. It keeps sensitive logic and database credentials away from the browser and enables shared, data-driven applications. Technologies include Java Servlets, PHP, and Ruby on Rails. Server-side validation is essential because client-side checks can be bypassed.

16. Analyze the architecture of Java Servlets with a detailed explanation and diagram.

Answer: A servlet application consists of a client/browser, web server, servlet container, servlet, and often a database or other resource. Flow: Browser → HTTP request → Web server/container → URL mapping selects servlet → service() (usually doGet/doPost) processes request and accesses resources → response object is generated → container/web server returns HTTP response → browser renders it. The container manages loading, initialization, request threads, and destruction. Servlets are Java-based and can use APIs such as JDBC.

17. Illustrate the process of generating dynamic content in Servlets with examples.

Answer: The browser sends an HTTP request to a servlet URL. The servlet container maps the URL, creates request/response objects, and invokes service(), which dispatches to doGet() or doPost(). The servlet reads parameters using methods such as getParameter(), applies logic or queries a database, and writes a response using the response writer. Example: in doGet(), read name = request.getParameter("name"); set content type to text/html; then write "<h1>Welcome " + escapedName + "</h1>". The container sends the generated response to the browser. User-provided output should be escaped to prevent injection.

18. Compare the types of cookies in Servlets and examine their role in session management.

Answer: A session/non-persistent cookie normally lasts only for the browser session and is removed when the browser session ends. A persistent cookie has an expiry or max-age and can remain across sessions until it expires or is deleted. In servlets, a Cookie can be added to a response and returned by the browser in later requests, helping identify a user or carry a session identifier. Cookies simplify state tracking because HTTP is stateless, but users may disable them and cookies should not contain sensitive data. Use Secure, HttpOnly, and appropriate SameSite attributes for session cookies.

19. Examine the life cycle of a Servlet with a detailed diagram and suitable example.

Answer: The servlet container manages the lifecycle. Flow: load servlet class → create one servlet instance → call init() once → call service() for each request (which dispatches to doGet/doPost) → call destroy() once before removing the servlet. init() initializes resources, service() handles requests and generates responses, and destroy() releases resources such as database connections. Example: a servlet initializes a configuration value in init(), responds with “Hello” in doGet(), and closes its resource in destroy(). Multiple requests may be handled by different threads, so shared mutable fields require thread safety.